Formação Completa

AWS Developer Associate

Automação de Testes e Infraestrutura como Código



Por que testar em ambientes isolados?

Proteção do ambiente produtivo

Evita impactos negativos aos usuários finais.

Detecção precoce de problemas

Identifica bugs antes de chegarem à produção.

Validação de código

Garante que as alterações funcionem conforme esperado.

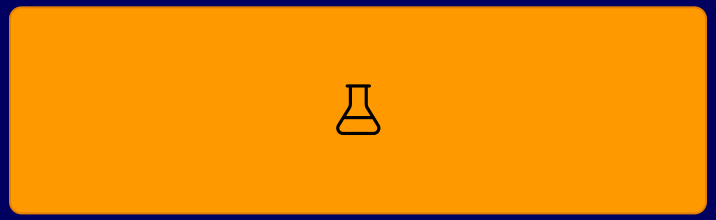


Conceito de Stages: Dev, Test, Prod



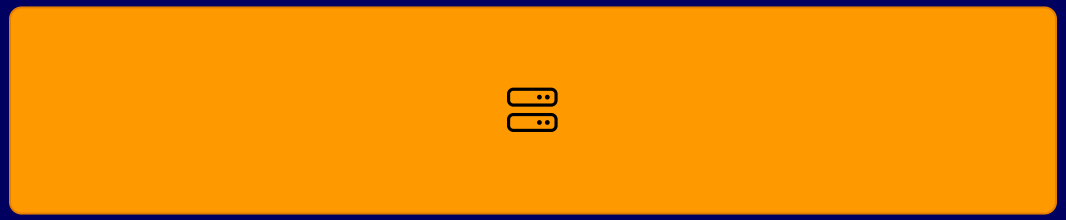
Desenvolvimento (Dev)

Ambiente para desenvolvimento inicial e testes rápidos.



Teste (Test)

Validação completa antes de ir para produção.



Produção (Prod)

Ambiente final para usuários reais.

Configuração de Variáveis por Stage

1

Definição

Crie variáveis específicas para cada ambiente.

2

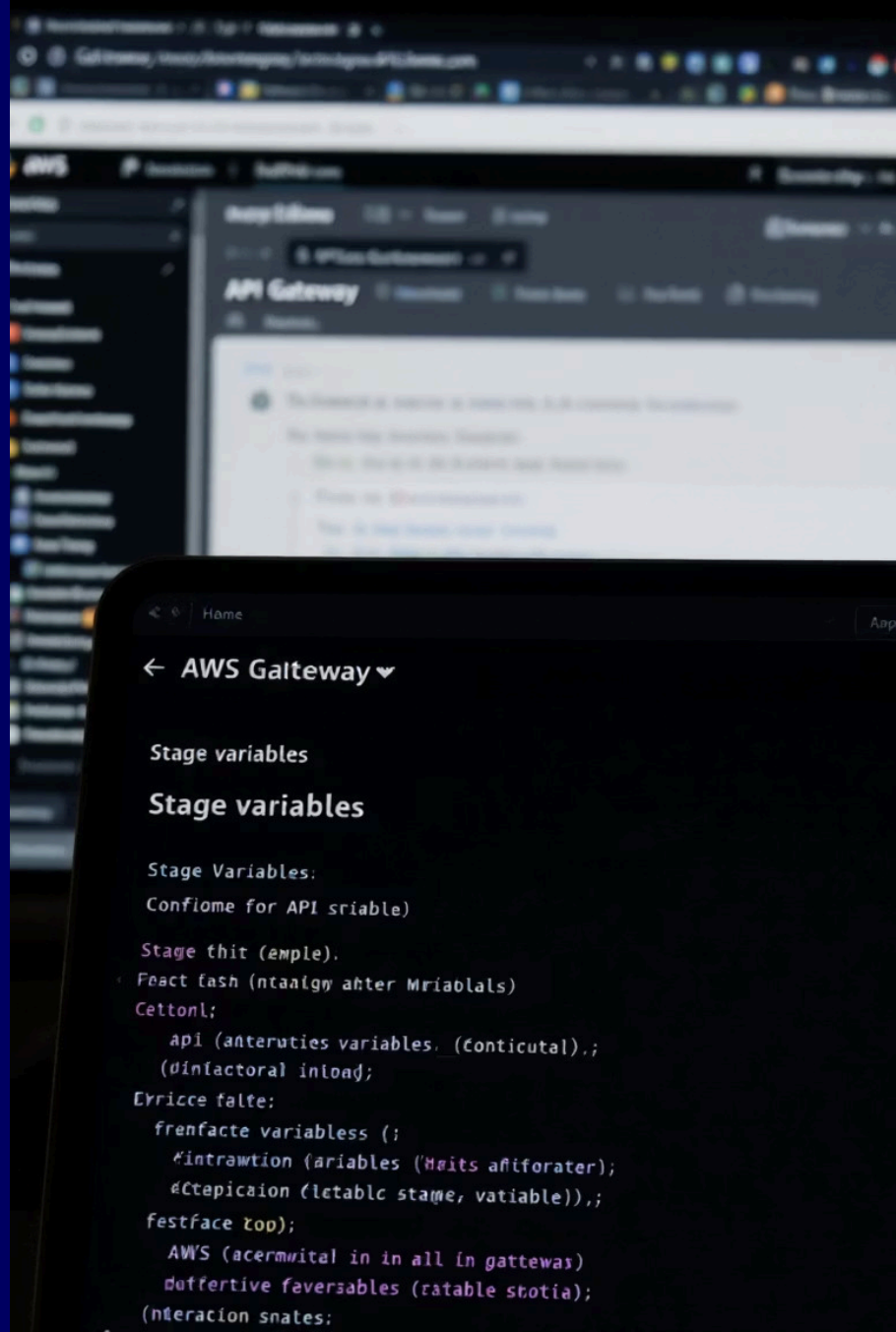
Implementação

Configure no API Gateway usando stageVariables.

3

Utilização

Acesse nas funções Lambda via context ou eventos.



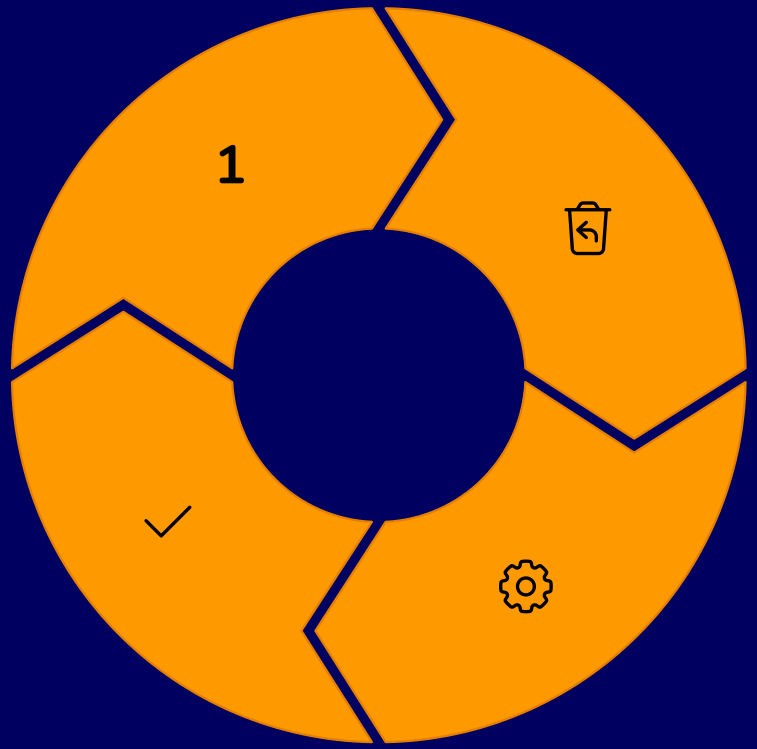
Rollback Facilitado via Isolamento

Detecção

Problema identificado em ambiente não-produtivo.

Validação

Nova solução testada antes de avançar.



Reversão

Rollback aplicado apenas no ambiente afetado.

Correção

Problema resolvido sem impactar produção.

CI/CD vs Deploy Manual

CI/CD Automatizado

- Consistência garantida
- Menos erros humanos
- Rastreabilidade completa
- Implantações mais rápidas

Deploy Manual

- Maior risco de erros
- Processos inconsistentes
- Dependência de pessoas
- Implantações mais lentas

Automação de Testes com CodeBuild

Configuração

Defina ambiente de build e dependências no buildspec.yml.

Execução

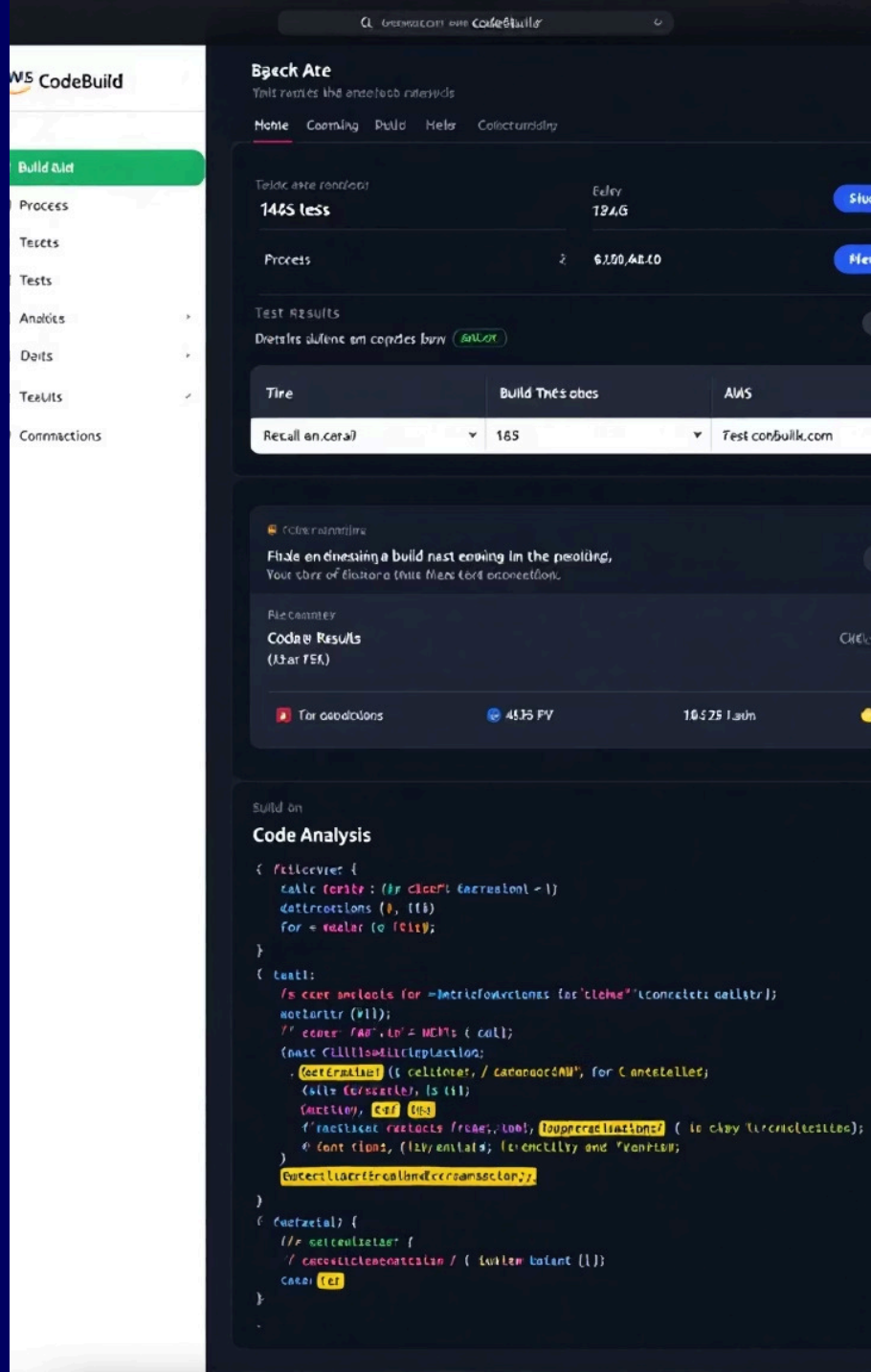
Rode testes automaticamente a cada commit ou pull request.

Relatórios

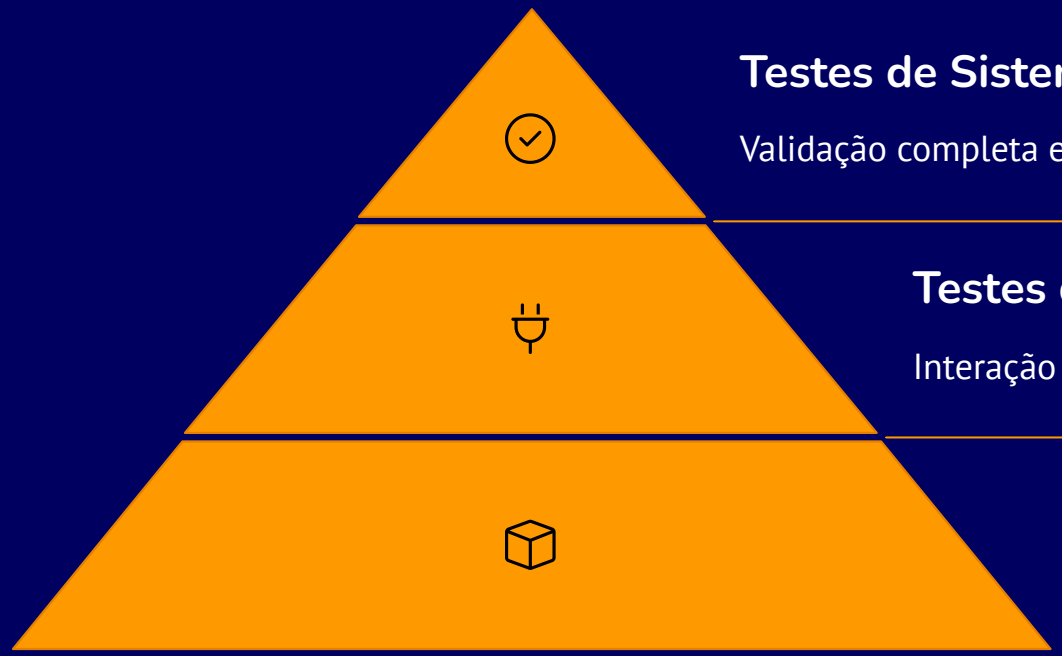
Gere relatórios de cobertura e resultados dos testes.

Integração

Conecte com CodePipeline para fluxo completo de CI/CD.



Tipos de Testes Automatizados



Testes de Sistema

Validação completa end-to-end

Testes de Integração

Interação entre componentes

Testes Unitários

Validação de componentes isolados

Relembre

Fluxo Completo de CI/CD

Commit

Desenvolvedor envia código para repositório.

Build

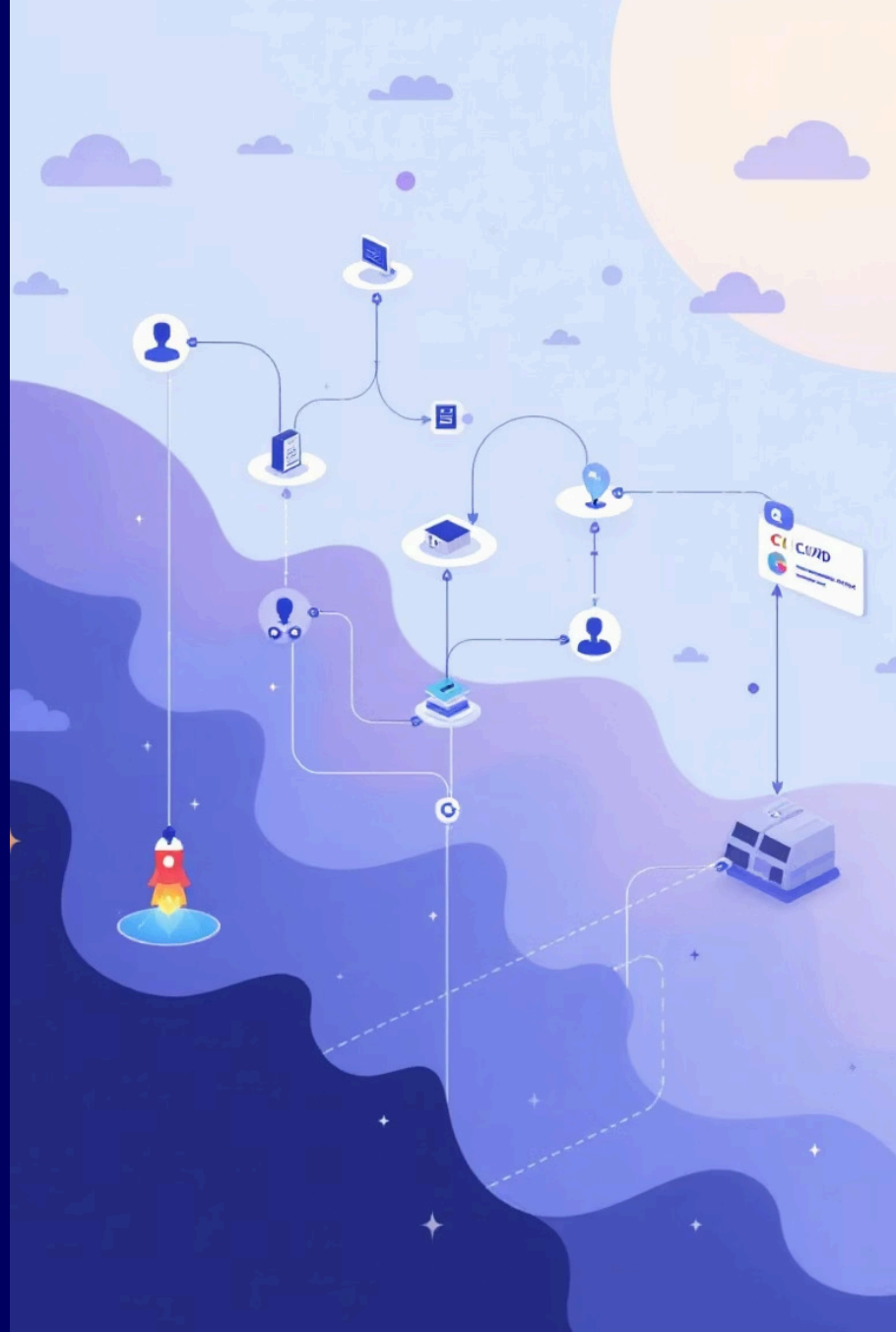
Sistema compila e empacota a aplicação.

Test

Execução de testes automatizados.

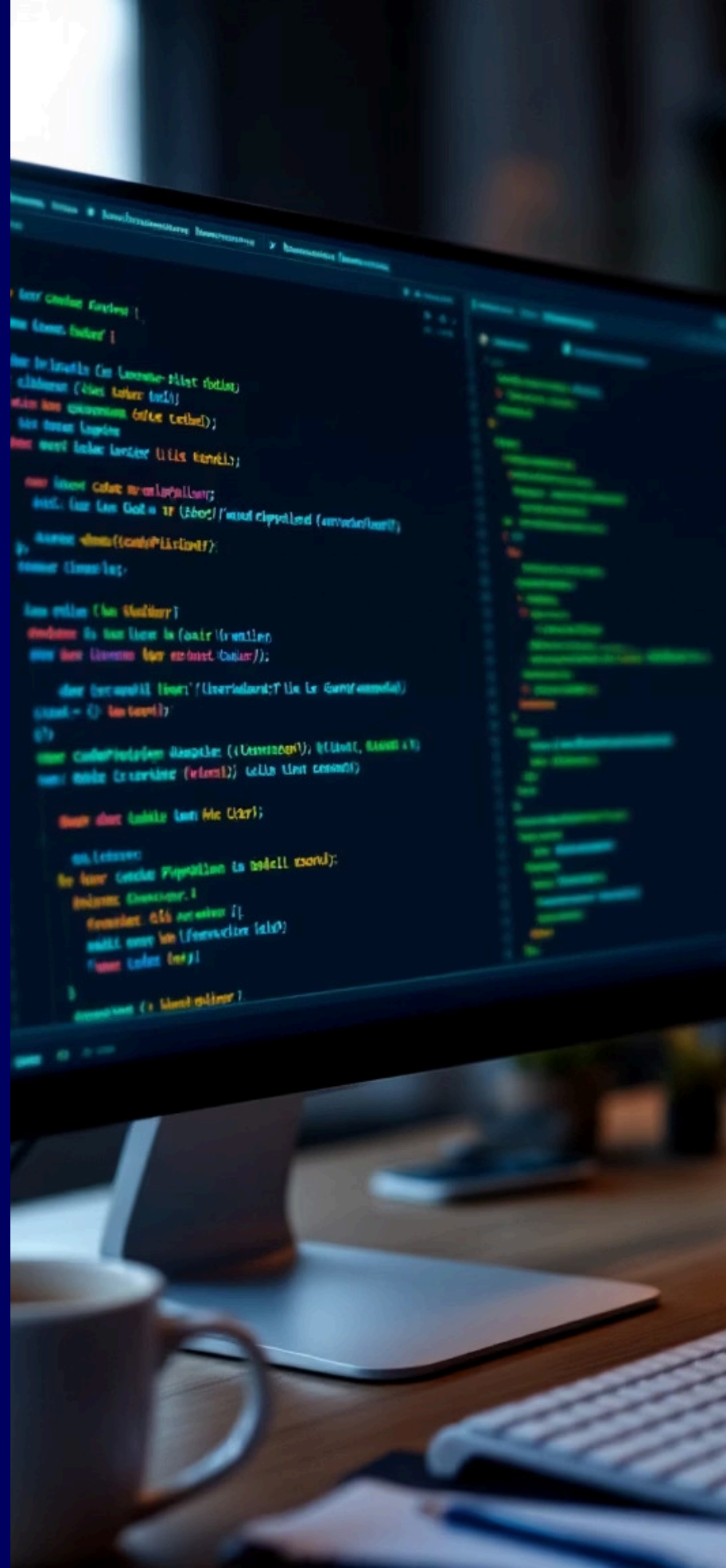
Deploy

Implantação em ambiente apropriado.



Exemplo de Pipeline com Etapas de Teste

```
{
  "pipeline": {
    "name": "MyAppPipeline",
    "stages": [
      {
        "name": "Source",
        "actions": [{"name": "Source", "provider": "CodeCommit"}]
      },
      {
        "name": "Build",
        "actions": [{"name": "BuildAndUnitTest", "provider": "CodeBuild"}]
      },
      {
        "name": "Test",
        "actions": [{"name": "IntegrationTest", "provider": "CodeBuild"}]
      },
      {
        "name": "Deploy",
        "actions": [{"name": "DeployToDev", "provider":
"CloudFormation"}]
      }
    ]
  }
}
```



Práticas Recomendadas para Testes

Testes Rápidos

Mantenha testes unitários abaixo de 10 minutos para feedback imediato.

Isolamento

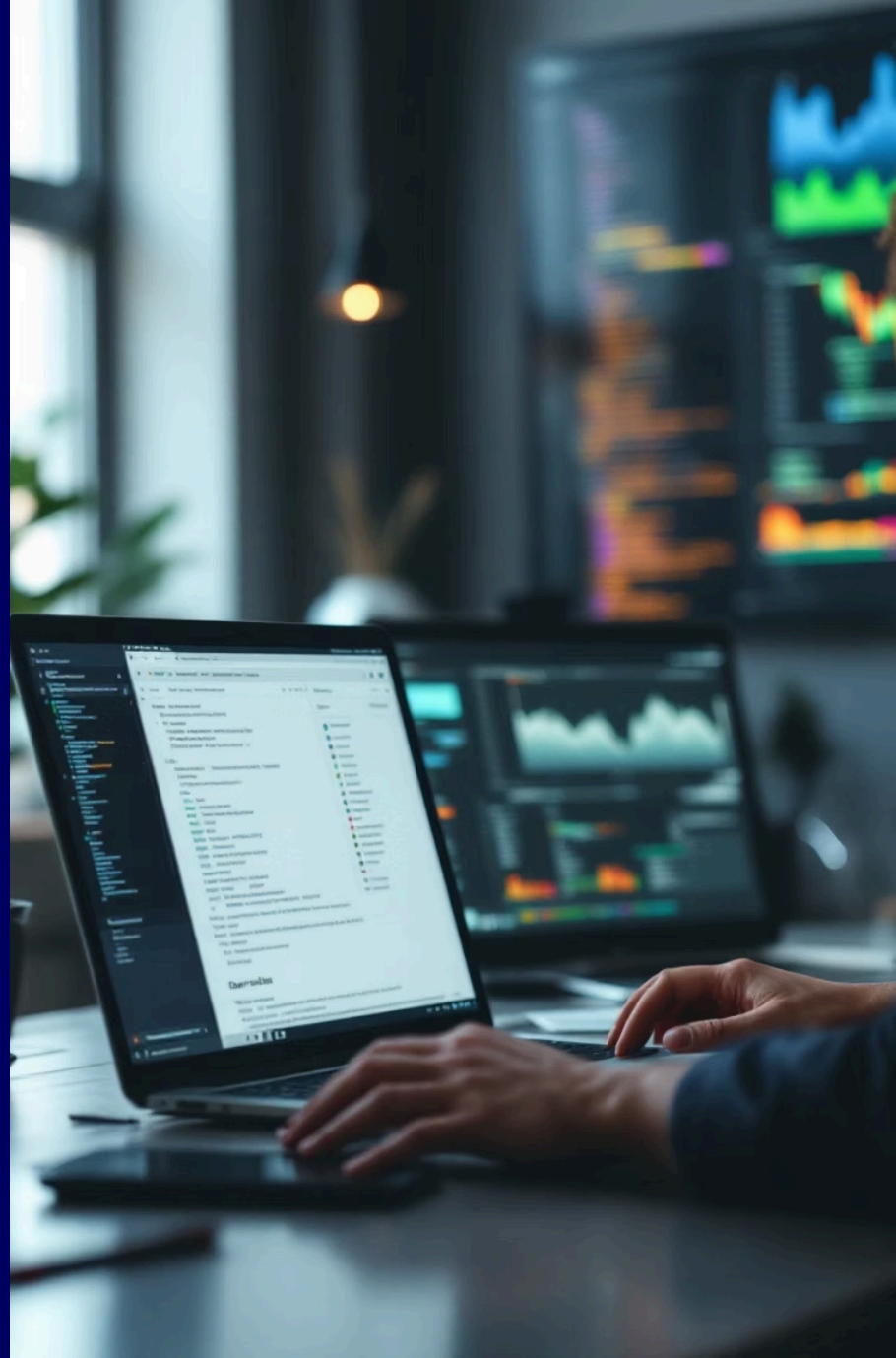
Isole dependências externas usando mocks ou stubs.

Cobertura

Monitore a cobertura de código, visando mínimo de 80%.

Dados de Teste

Use dados consistentes e isolados para cada execução.





Desafios Comuns e Mitigações

Testes lentos	Paralelização e otimização de recursos
Testes instáveis	Isolamento de ambiente e retry automatizado
Dependências externas	Mocks, stubs e containers locais
Manutenção complexa	Padrões de teste e frameworks consistentes

Execução da função: bem-sucedida (logs)

▼ Detalhes


```
{
  "statusCode": 200,
  "headers": {
    "Content-Type": "application/json",
    "Access-Control-Allow-Origin": "*"
  },
  "body": "{\"message\": \"Ola do ambiente de PRODUCAO!\", \"functionVersion\": \"$LATEST\", \"functionAlias\": \"minha-funcao-proxy-lab-seunome\"}"
}
```


Por que Versionar Funções Lambda?

Rastreabilidade

Histórico completo de alterações e implantações.

Rollback Seguro

Retorno rápido a versões anteriores funcionais.

Implantação Gradual

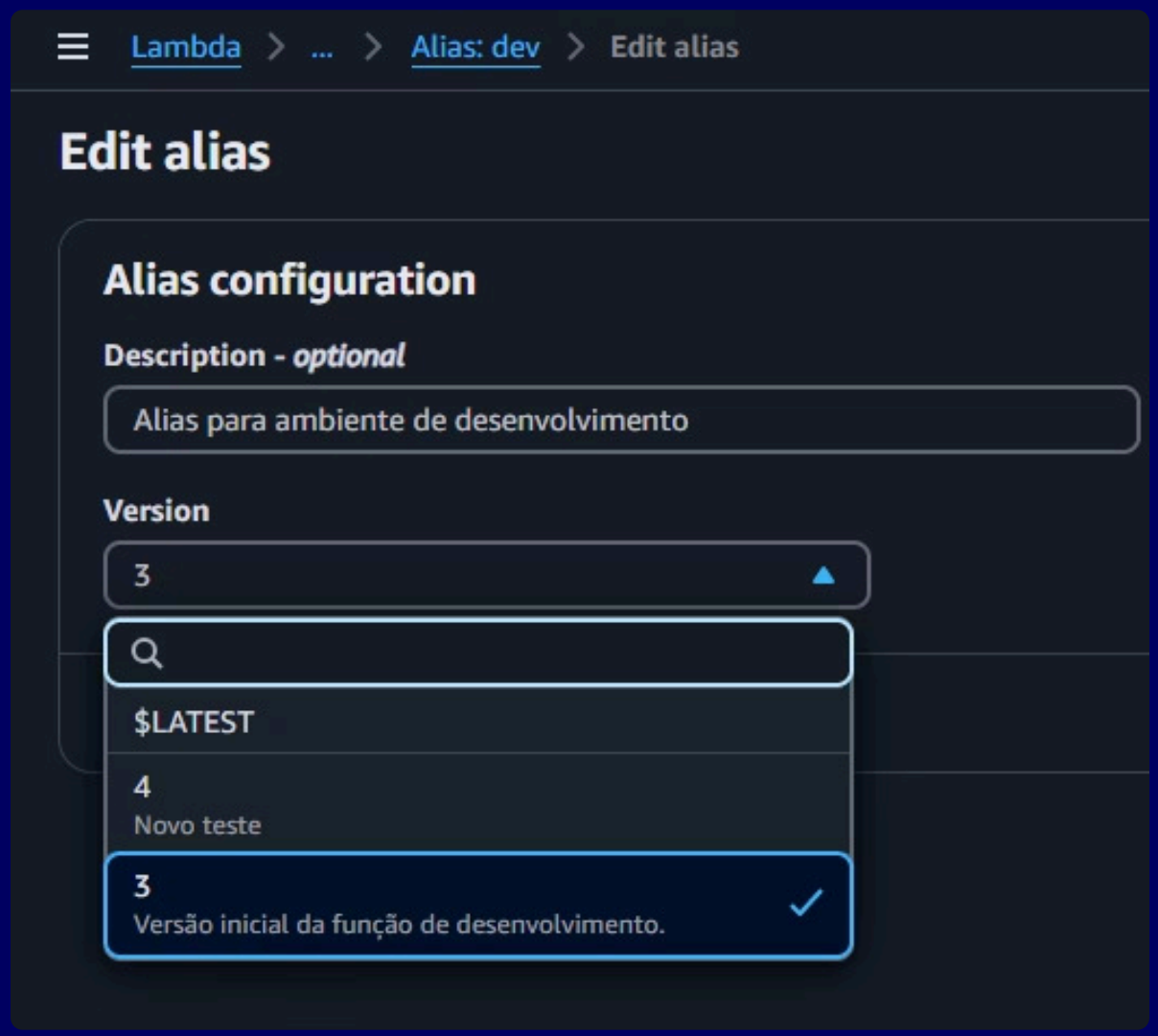
Possibilidade de testes A/B e canary (falarrmos mais sobre canary posteriormente) deployments.

Segurança

Controle preciso sobre qual código está em execução.

Name	Versions	Description
dev	version: 3 (weight=100%)	Alias para ambiente de desenvolvimento

Conceito de Alias no Lambda



1

Definição

Ponteiros para versões específicas de funções Lambda.

2

Criação

Crie aliases como dev, test, prod apontando para versões.

3

Atualização

Mude o alias para apontar para novas versões quando validadas.

Deploy Seguro com Versionamento

Publicar Nova Versão

Crie uma versão imutável do código atual.

Testar em Ambiente Isolado

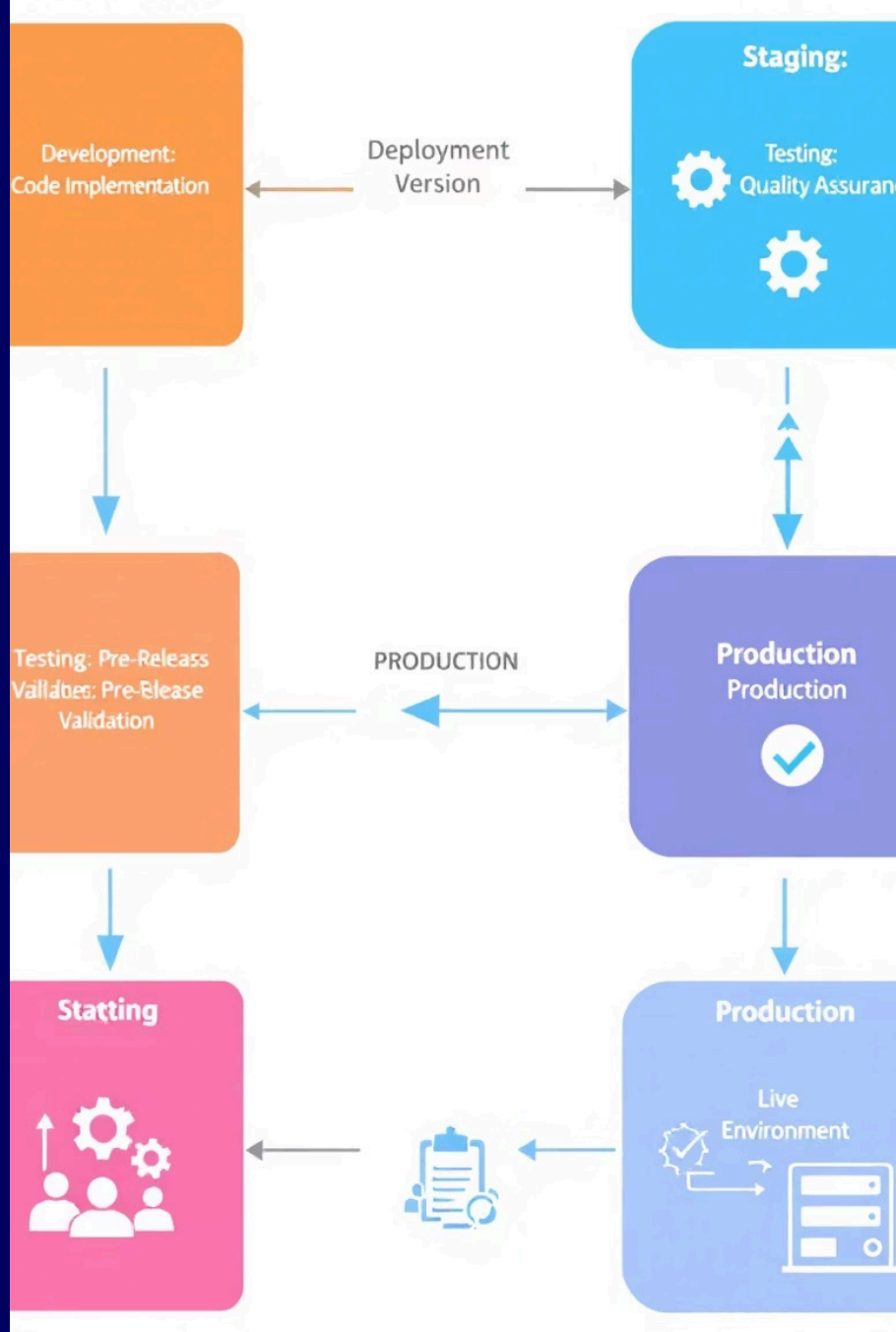
Aponte alias "dev" ou "test" para nova versão.

Validar Funcionamento

Execute testes automatizados contra o novo código.

Promover para Produção

Atualize alias "prod" para apontar para versão validada.



Relembre: Laboratório 6

- Publicar diferentes versões da função Lambda para representar estados distintos do código.
- Criar Aliases no Lambda (ex: dev, prod) que funcionam como ponteiros para versões específicas da função.
- Criar uma API REST no API Gateway.
- Configurar a integração do tipo Proxy entre a API Gateway e a função Lambda.
- Criar Stages no API Gateway (ex: Desenvolvimento, Producao) para representar os diferentes ambientes de implantação da API.
- Integrar cada Stage da API Gateway com o Alias correspondente da função Lambda, garantindo que cada ambiente da API invoque a versão correta da função.

Como faríamos usando código?

Publicação de nova versão da função Lambda

```python
import boto3

lambda_client = boto3.client('lambda', region_name='us-east-1')

response = lambda_client.publish_version(
    FunctionName='MinhaFuncaoLambda',
    Description='Versão estável com novos recursos'
)

new_version = response['Version']
print("Nova versão publicada:", new_version)
```

Criação/atualização de alias para apontar para a nova versão

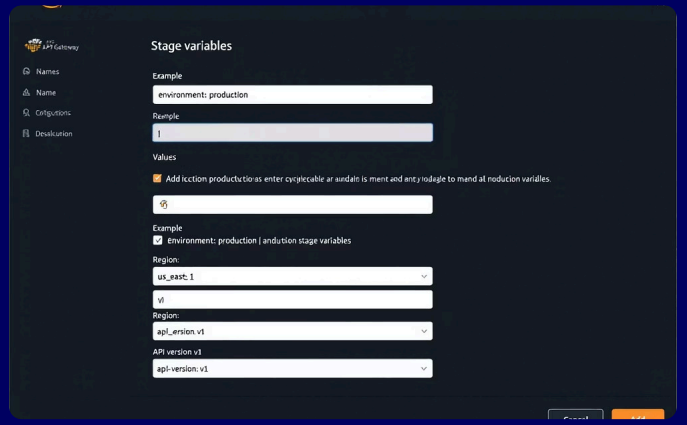
```python
alias_response = lambda_client.update_alias(
    FunctionName='MinhaFuncaoLambda',
    Name='prod',  # ou 'dev'
    FunctionVersion=new_version,
    Description='Alias para ambiente de produção'
)

print("Alias 'prod' apontando para a versão:", alias_response['FunctionVersion'])
```

Configuração do API Gateway com integração Proxy e variável de estágio (YAML)

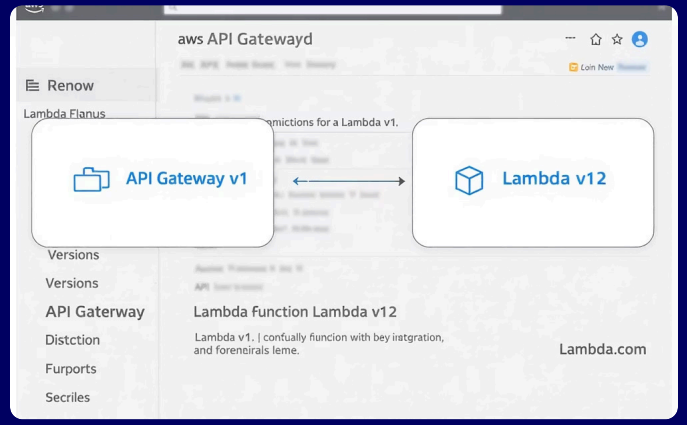
```yaml
Integration:
  Type: AWS_PROXY
  Uri: arn:aws:apigateway:${AWS::Region}:lambda:path/2015-03-31/functions/
       arn:aws:lambda:${AWS::Region}:${AWS::AccountId}:function:
       MinhaFuncaoLambda:${stageVariables.ENV}/invocations
  HttpMethod: POST
```

Integração com API Gateway



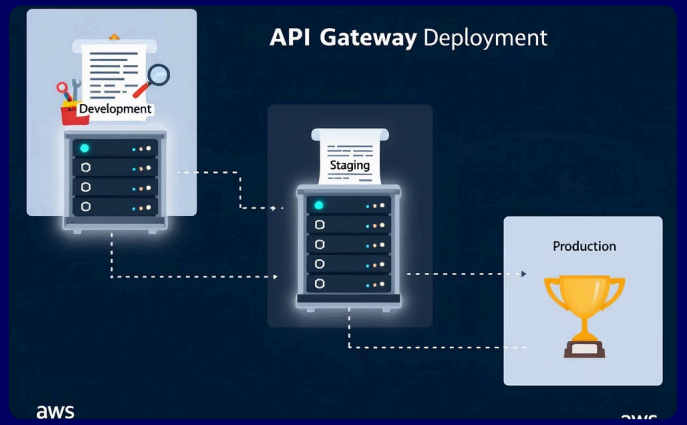
Configuração de Stage Variables

Defina variáveis como lambdaAlias: prod, dev ou test.



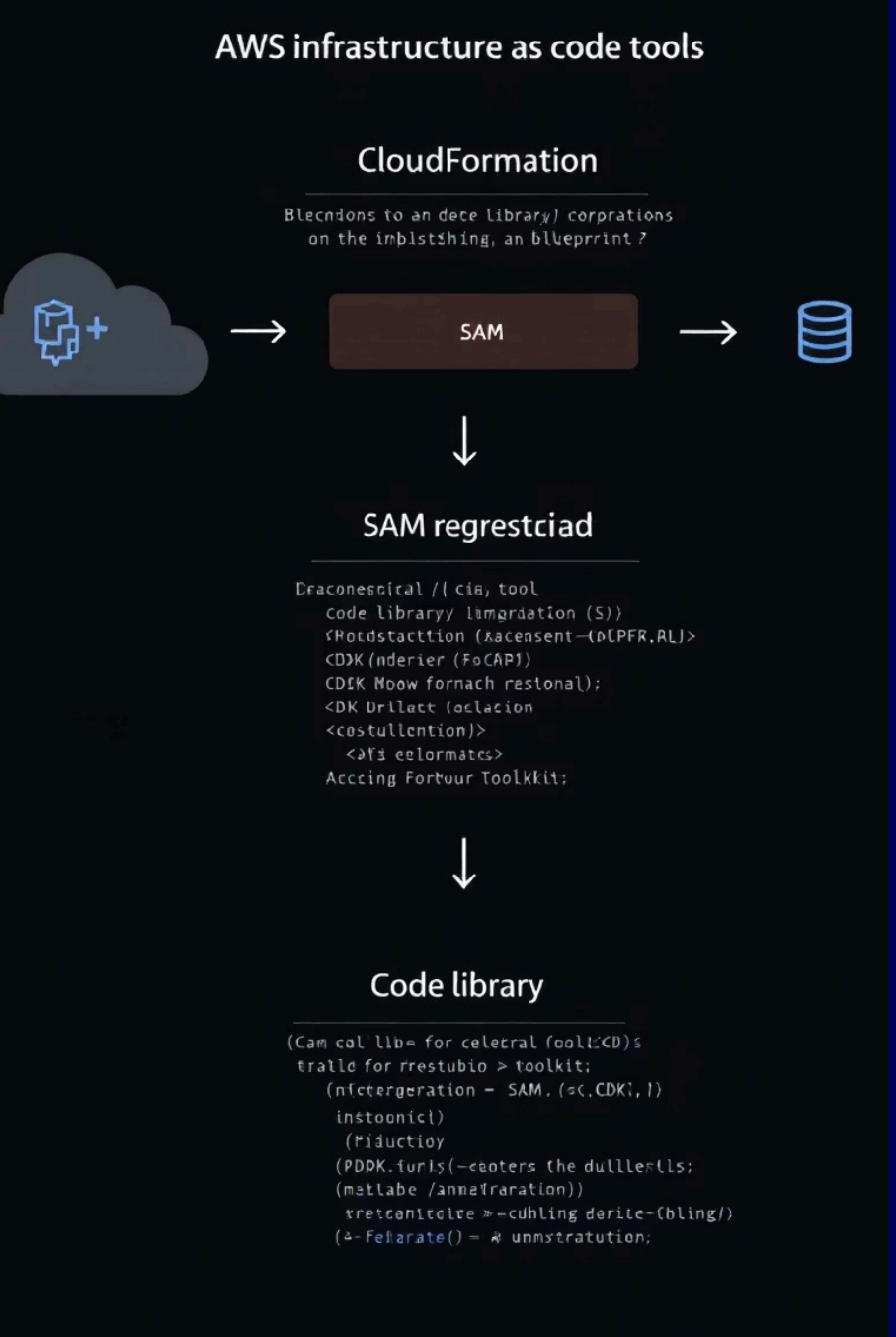
Integração com Lambda

Use ${stageVariables.lambdaAlias} na configuração do Lambda.



Implantação em Múltiplos Stages

Cada stage aponta para o alias correspondente.



Comparativo: CloudFormation vs SAM vs CDK

CloudFormation	Verboso, JSON/YAML, baixo nível, completo
SAM	Simplificado, YAML, foco em serverless, macros CF
CDK	Código (TypeScript/Python), orientado a objetos, construtos



Quando Escolher CDK

Times de Desenvolvedores

Preferência por linguagens de programação em vez de YAML/JSON.

Projetos Complexos

Necessidade de lógica, loops e condicionais avançadas.

Reutilização

Criação de componentes personalizados e bibliotecas compartilhadas.

Validação

Verificação de tipos e validação em tempo de compilação.

Workflow do CDK

Desenvolvimento

Escreva código em TypeScript, Python, Java ou C#.

Synth

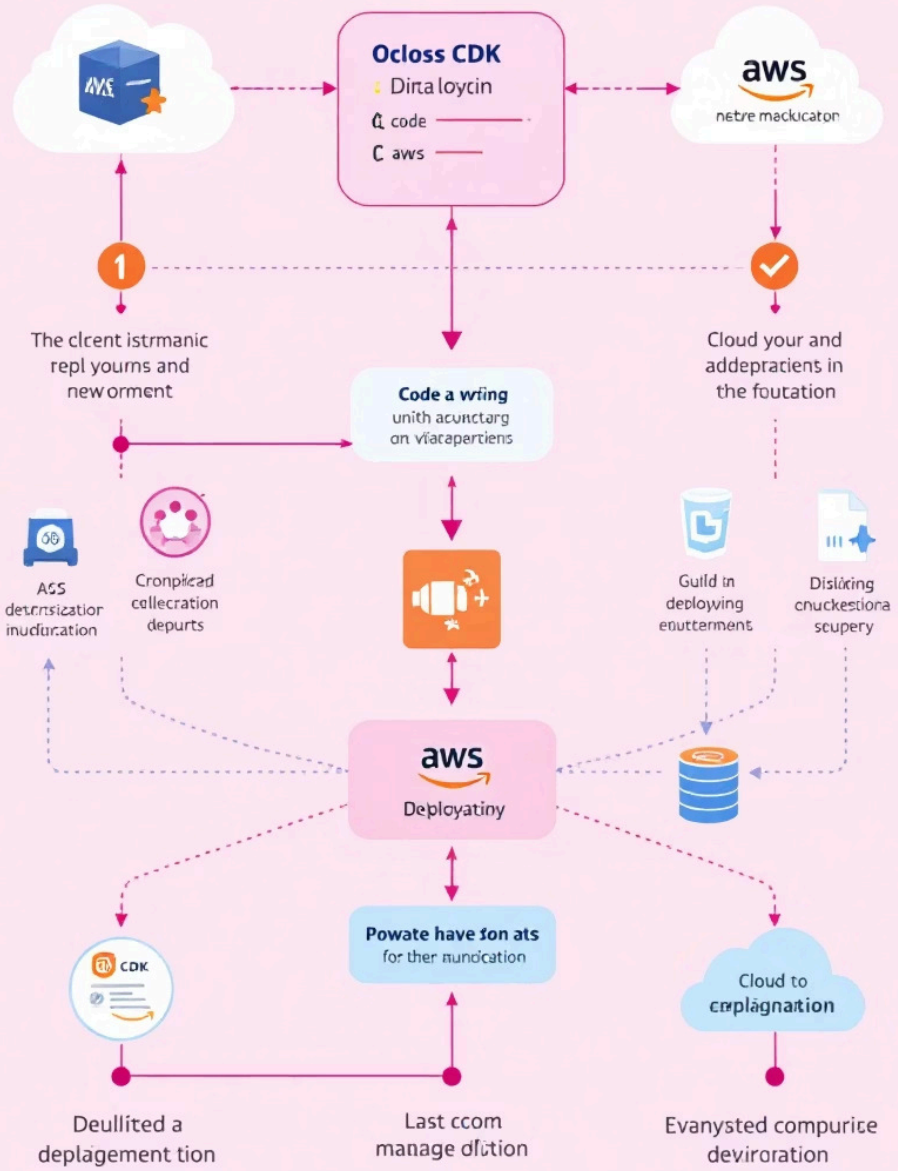
Gere template CloudFormation com "cdk synth".

Diff

Compare mudanças com "cdk diff" antes de implantar.

Deploy

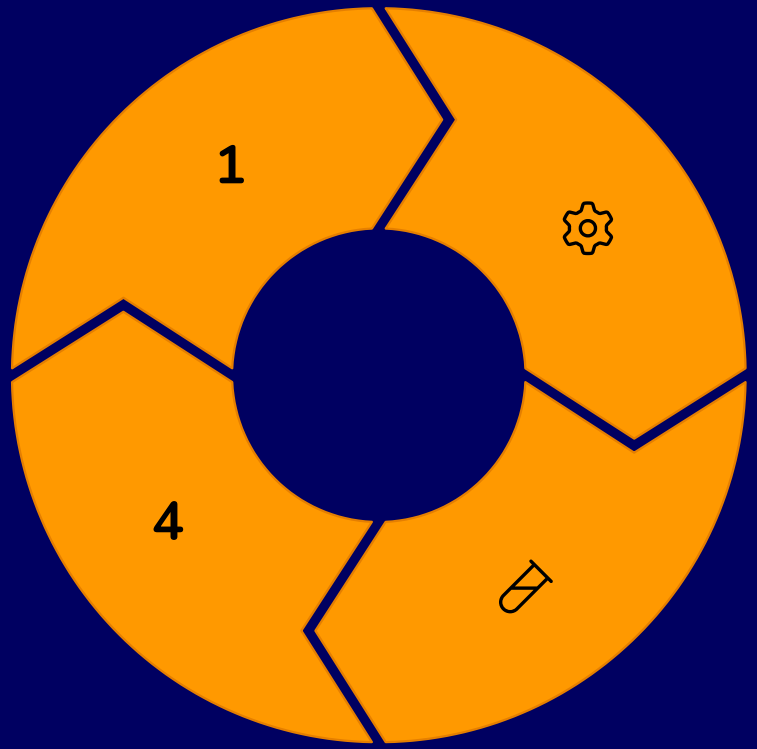
Implante infraestrutura com "cdk deploy".



Integração do CDK com Git e CodePipeline

Commit no Git

Código CDK versionado no repositório.



Build

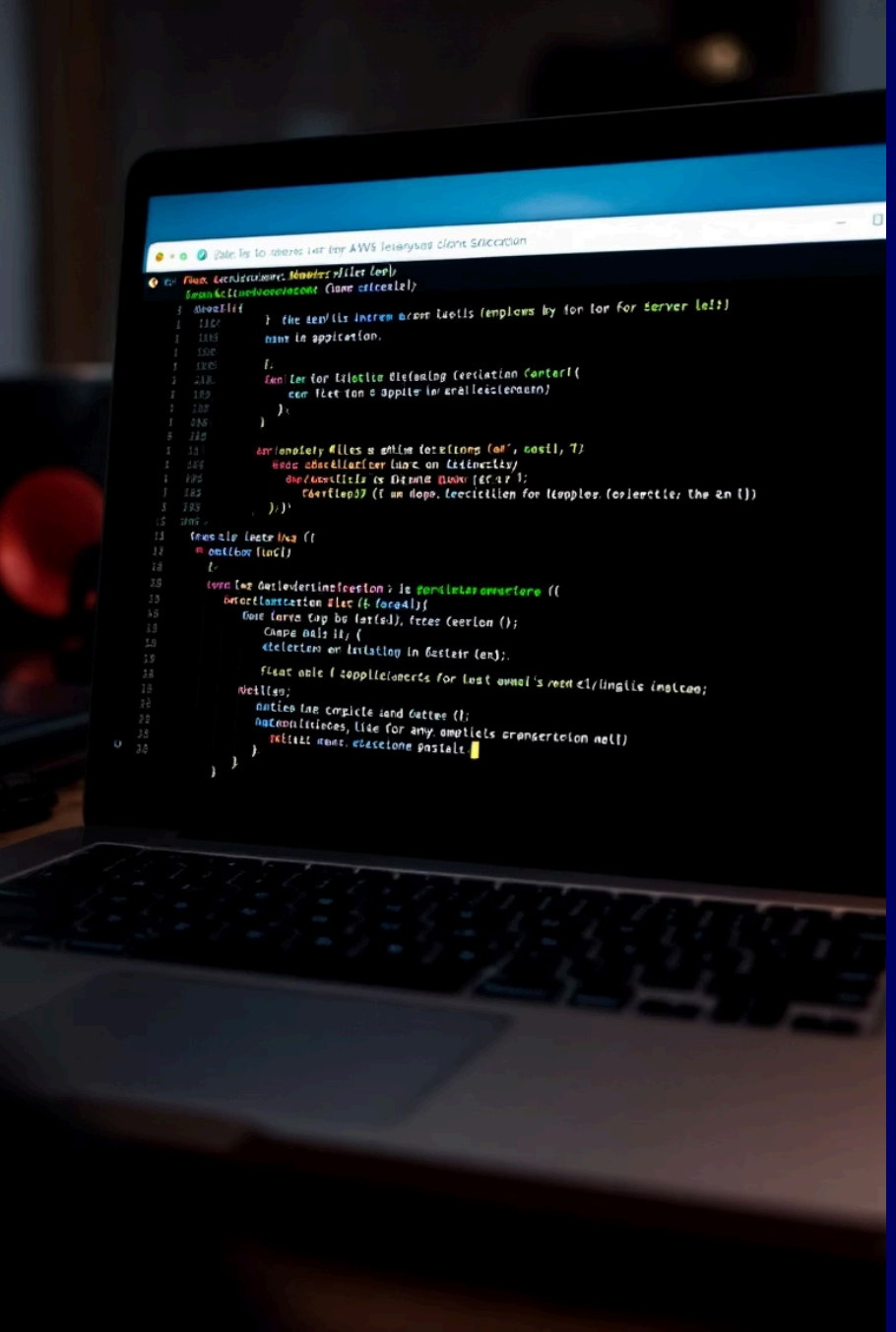
CodeBuild executa "npm install" e "cdk synth".

Teste

Validação do template gerado e testes unitários.

Deploy

CodePipeline implanta com "cdk deploy".



AWS SAM CLI - Simulação Local

Instalação

Instale Docker e SAM CLI para simulação local.

Invocação Local

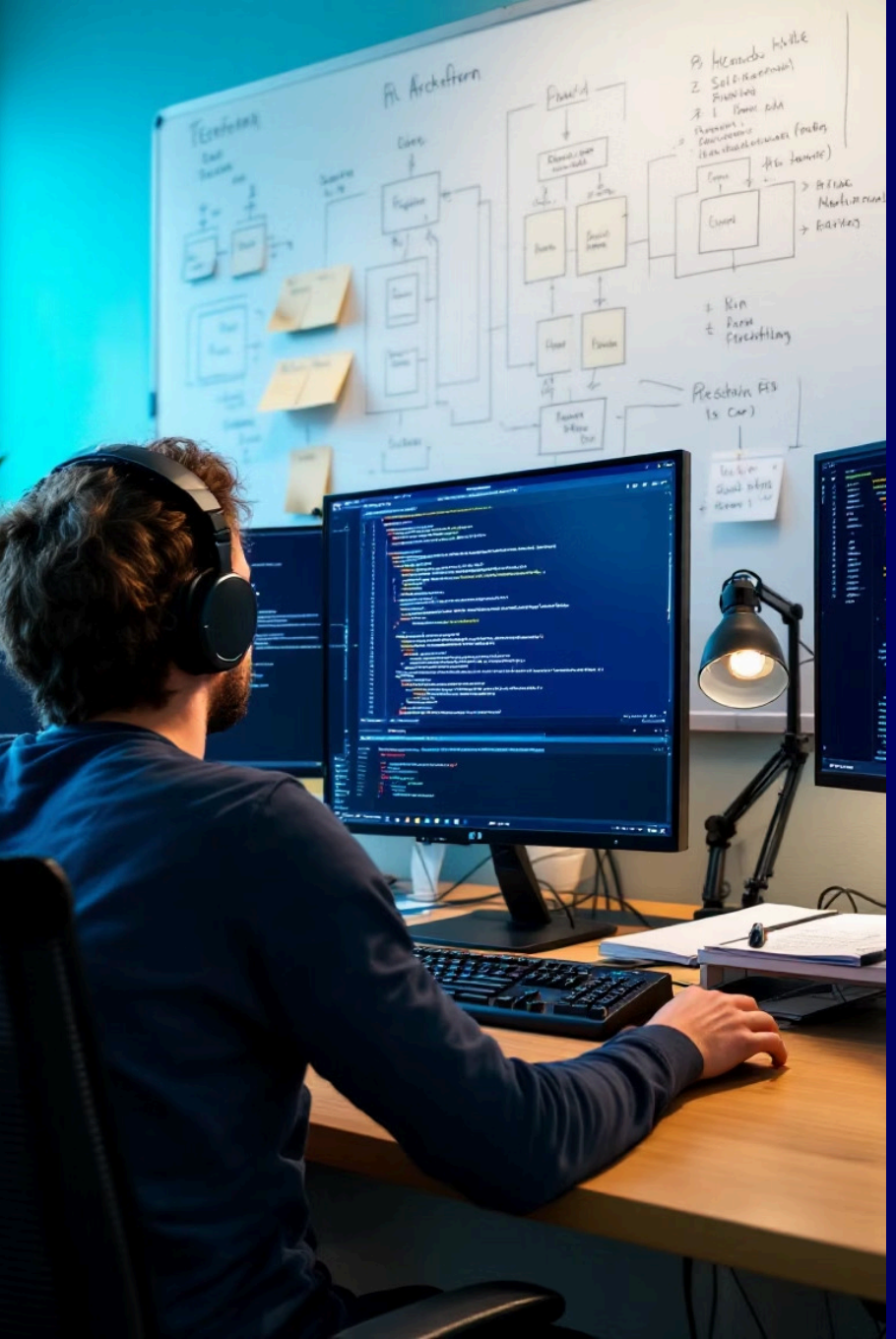
Use "sam local invoke" para testar funções isoladamente.

API Local

Execute "sam local start-api" para simular API Gateway.

Testes Integrados

Automatize testes contra a API local.



Boas Práticas com Templates

Modularização

Divida templates grandes em componentes menores e reutilizáveis.

Tagging

Use tags consistentes para faturamento e organização.

Segurança

Aplique princípio de menor privilégio em todas as permissões.

Documentação

Documente parâmetros e recursos com comentários claros.

Erros Comuns com SAM/CDK

Permissões Insuficientes

IAM roles sem as permissões necessárias para recursos.

Dependências Circulares

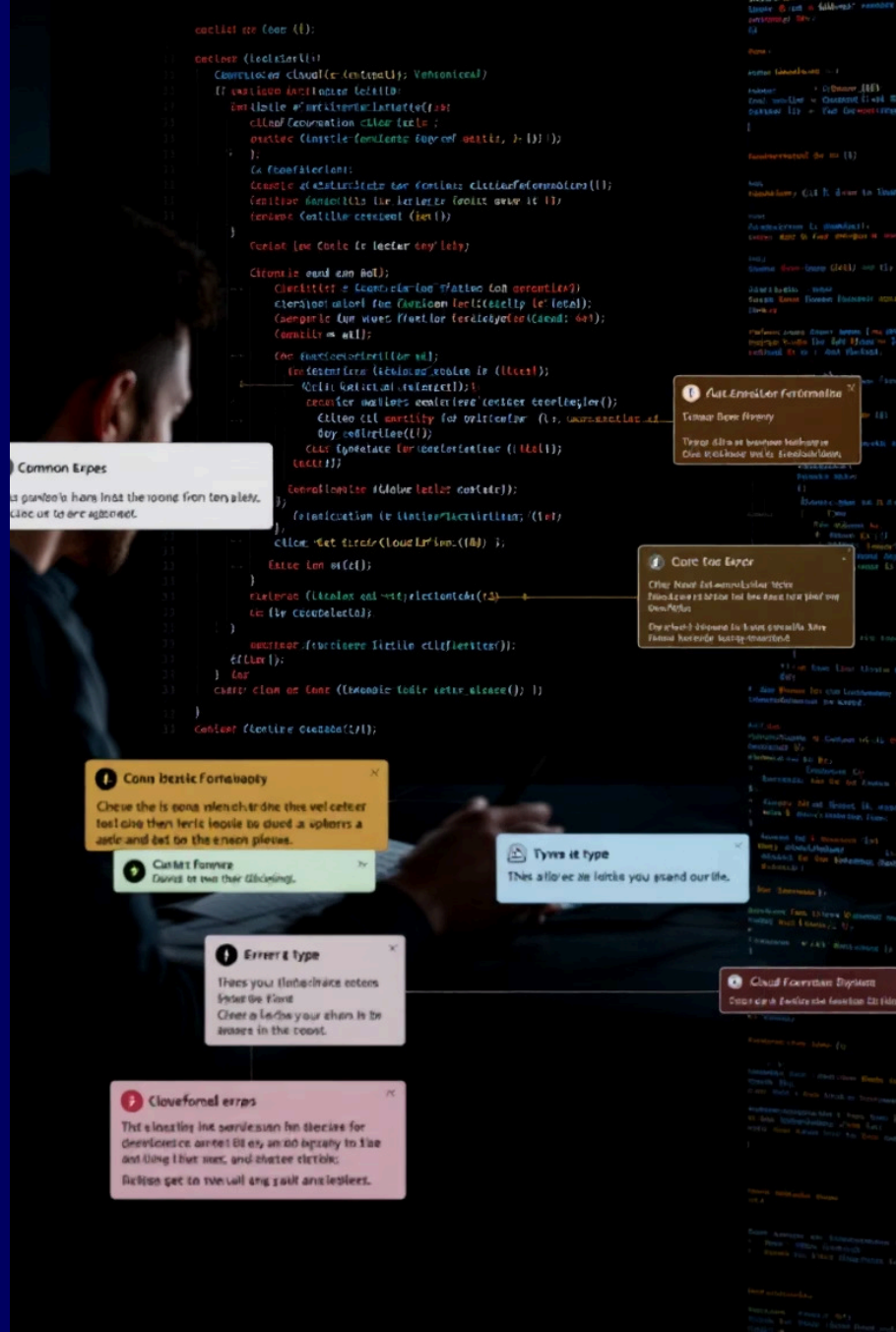
Recursos que dependem mutuamente um do outro.

Limites de Serviço

Exceder quotas de serviço durante implantação.

Drift de Configuração

Mudanças manuais causando inconsistências com o template.



```
echo "🔄 Validando template..."
aws cloudformation validate-template --template-body file://$TEMPLATE_FILE

echo "🚀 Fazendo deploy do stack $STACK_NAME..."
aws cloudformation deploy \
```

Templates como Código Versionado

Histórico Completo

Rastreie todas as mudanças na infraestrutura ao longo do tempo.

Colaboração

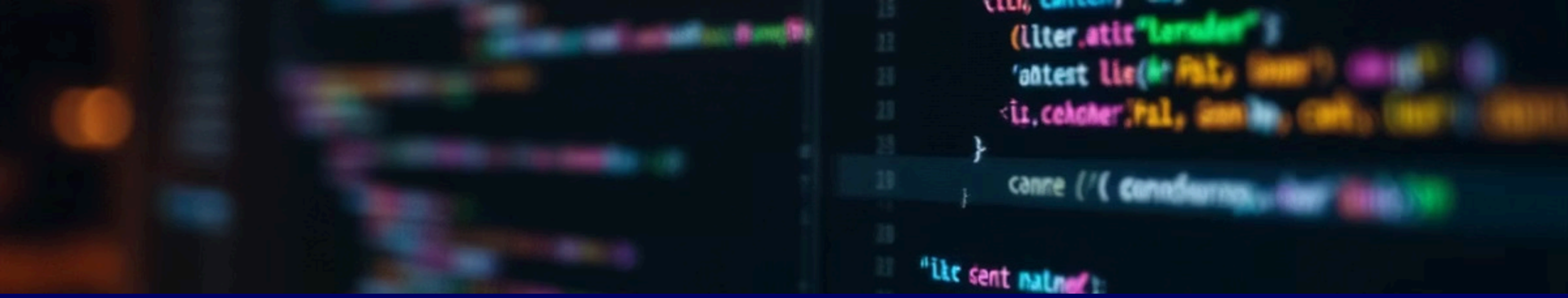
Permita que múltiplos desenvolvedores trabalhem nos mesmos templates.

Segurança

Implemente revisão de código para mudanças de infraestrutura.

Reversão

Volte facilmente a estados anteriores conhecidos.



Exemplo de Git com CloudFormation

```
# Clonar repositório de infraestrutura
git clone https://github.com/empresa/infra-templates.git

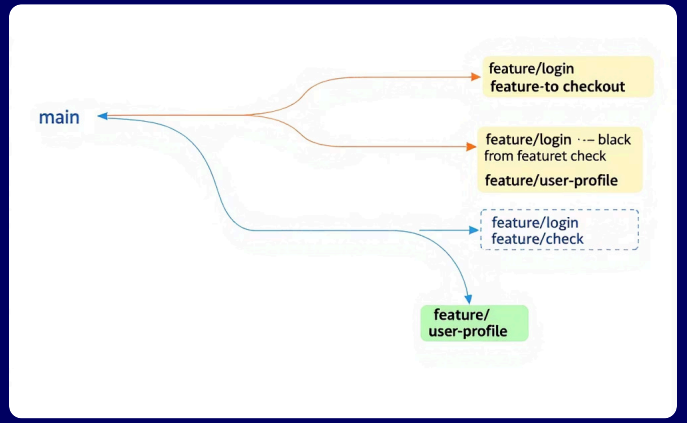
# Criar branch para nova feature
git checkout -b feature/nova-lambda

# Editar template
vim templates/lambda-stack.yaml

# Commit das mudanças
git add templates/lambda-stack.yaml
git commit -m "Adiciona nova função Lambda para processamento de imagens"

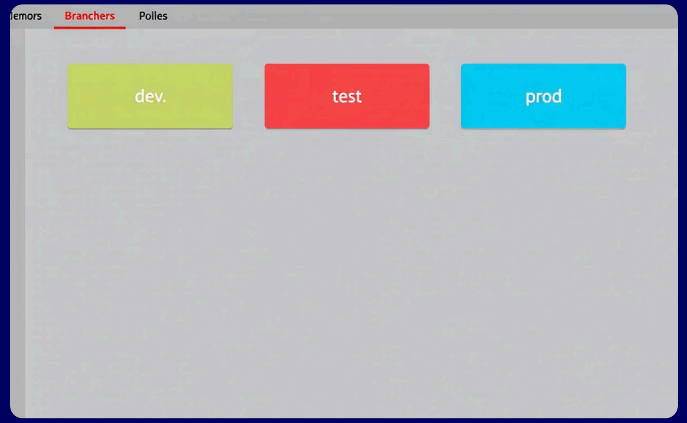
# Push e pull request
git push origin feature/nova-lambda
```

Branching para Infraestrutura



Feature Branches

Crie branches para cada nova funcionalidade ou mudança.



Environment Branches

Mantenha branches específicas para cada ambiente (dev/test/prod).



Pull Requests

Exija revisão de código antes de mesclar mudanças.

Auditoria e Rastreabilidade

Benefícios do Versionamento

- Histórico completo de mudanças
- Identificação de autores
- Timestamps precisos
- Comentários explicativos
- Comparação entre versões

Integração com AWS Config

- Registro de mudanças em recursos
- Validação de conformidade
- Alertas de desvios
- Histórico de configuração

Rollback de Infraestrutura com Git

Identificar Commit Estável

Localize o último estado conhecido funcional no histórico.

Criar Branch de Recuperação

Crie branch a partir do commit estável identificado.

Implantar Versão Anterior

Execute o pipeline com a versão estável do template.

Validar Funcionamento

Confirme que o sistema voltou ao estado operacional.

```
#!/bin/bash
set -e

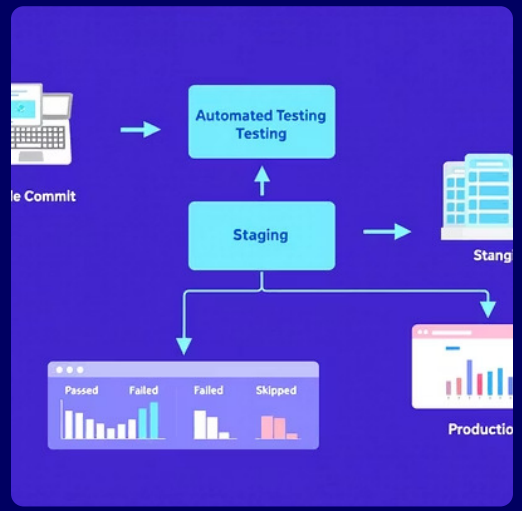
STACK_NAME="vpc-stack"
TEMPLATE_FILE="templates/vpc.yaml

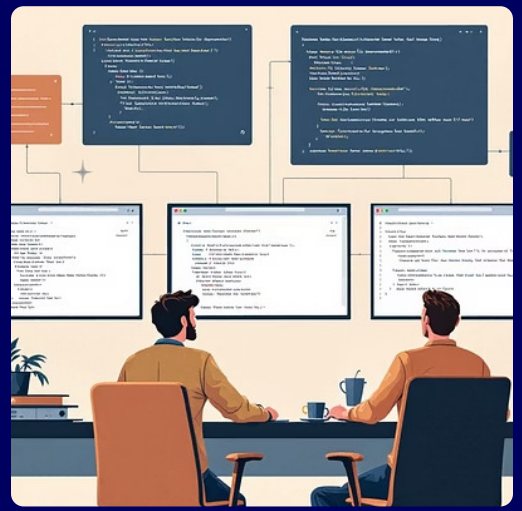
echo "🔄 Validando template..."
aws cloudformation validate-templ

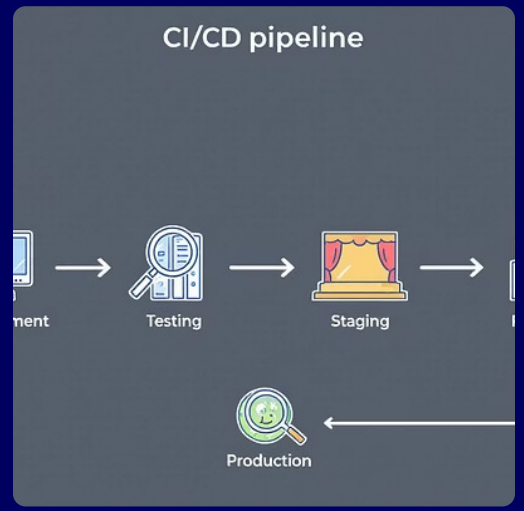
echo "🚀 Fazendo deploy..."
aws cloudformation deploy \
  --stack-name $STACK_NAME \
  --template-file $TEMPLATE_FILE
  --capabilities CAPABILITY_NAMED

echo "✅ Deploy concluído."
```

Práticas de Excelência e Qualidade









Combine automação de testes, versionamento de infraestrutura, ambientes isolados, revisão de código e monitoramento contínuo.

Conceitos Mais Cobrados no Exame

CI/CD Pipeline

Foco em automação de build, teste e deploy.

Serverless

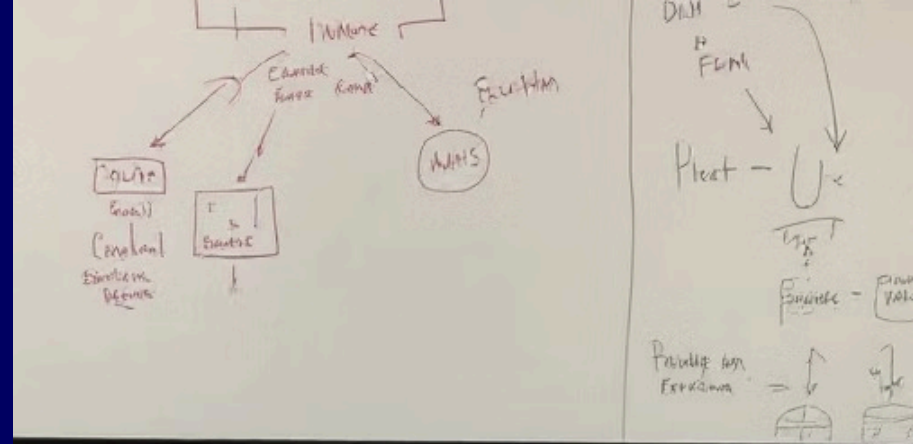
Lambda, API Gateway e integração com outros serviços.

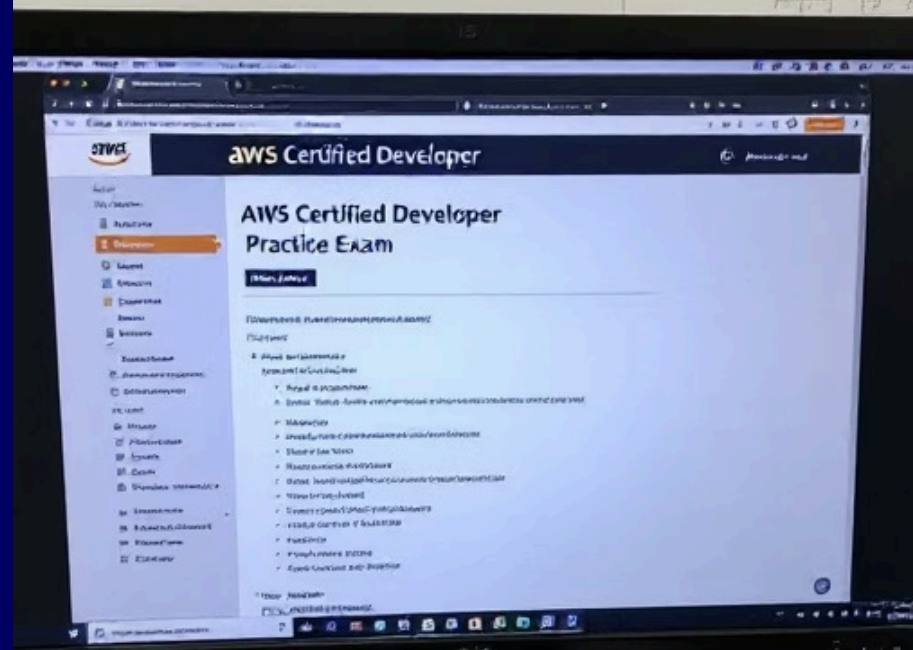
IaC

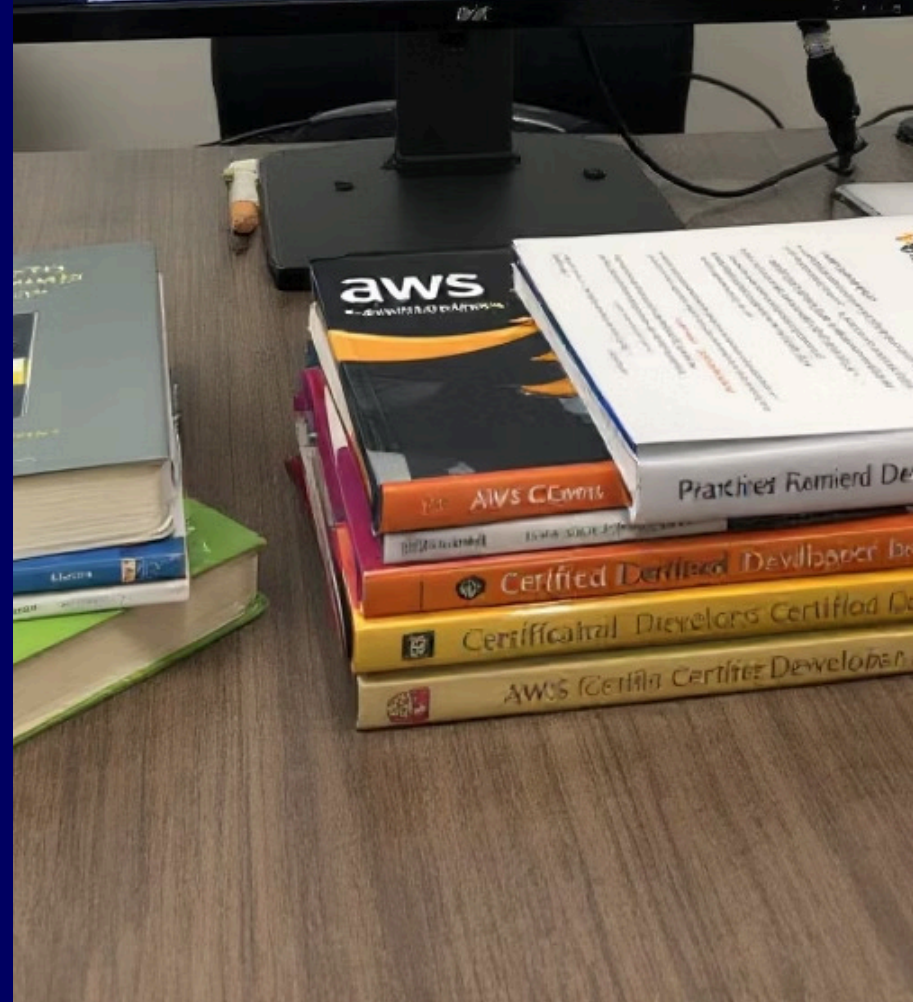
CloudFormation, SAM e melhores práticas.

Monitoramento

CloudWatch, X-Ray e troubleshooting.







Por que isso é Importante (Visão Prática e da Certificação)

Consistência de Ambientes

Ambientes idênticos reduzem erros e facilitam rollbacks. A infraestrutura como código (IaC) garante provisionamento consistente em todos os estágios.

Automação e CI/CD

Pipelines automatizados garantem integração, testes e deploy seguros. Isso aumenta confiabilidade e acelera o ciclo de desenvolvimento.

Gerenciamento de Versões

Git e ferramentas similares permitem rastrear mudanças e recuperar versões estáveis. Facilita colaboração entre equipes e manutenção contínua.

Relevância para Certificação

Conceitos fundamentais para o exame AWS Certified Developer – Associate. Essenciais também para ambientes de produção modernos.

A combinação de consistência, automação e versionamento cria uma base sólida para operações confiáveis e eficientes na AWS.